



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506 / SE ZG506

Lecture No. 2 and 3

Agenda



- Modern application requirements
- Cloud-native evolution
- Introducing Cloud-native software
- Cloud-enabled vs Cloud-based vs Cloud-native apps
- Examples for Cloud-native apps
- CNCF Landscape
- Overview of Cloud-native ecosystem
 - Microservices and Service Mesh
 - Serverless Computing and Serverless Stack
 - DevOps and GitOps
 - Containers and Kubernetes
- Case Study: Analysis of a cloud-native application architecture



Modern Application Requirements

Ref: “Cloud Native Patterns”, by Cornelia Davis. Publisher: Manning, 2019

Modern Application Requirements

Streaming / OTT



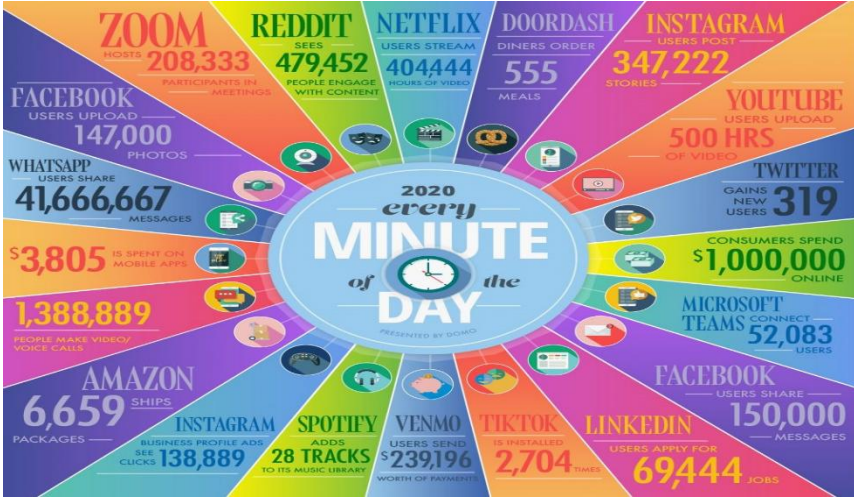
Social Media Apps



E-Commerce Apps



Data Deluge (Infographic)



Modern Application Requirements

General Requirements

1. Zero Downtime
2. Shortened Feedback Cycles
3. Mobile and Multi-device support
4. Data driven

Software Quality Attributes

- Scalability
- Performance
- Availability
- Reliability
- Interoperability
- Testability
- Usability
- Modifiability
- Security
- Portability
- Maintainability

COMPANY	DEPLOY FREQUENCY	DEPLOY LEAD TIME	RELIABILITY	CUSTOMER FEEDBACK
AMAZON	23,000/day	minutes	high	high
GOOGLE	5,500/day	minutes	high	high
NETFLIX	500/day	minutes	high	high
FACEBOOK	1/day	hours	high	high
TWITTER	3/week	hours	high	high
TYPICAL ENTERPRISE	once every 9 months	months or more	low/medium	low/medium

From "The Phoenix Project"

Shift in the last 6-7 years



1. Shift from Monolith **to** **Microservices Architecture**
2. Shift from On-premise **to** **Cloud (AWS, Azure, GCP)** for hosting applications
3. Shift from Virtual Machine based deployment **to** **Container based deployment**
4. Shift from Traditional CI/CD approaches (Monolith, Cloud-based or Cloud enabled apps, VM based deployment) **To** **Cloud-native CI/CD [Microservices, Docker/ Kubernetes]**

Evolution

	Development Process	Application Architecture	Deployment & Packaging	Application Infrastructure
~ 1980	Waterfall	Monolithic	Physical Server	Datacenter
~ 1990				
~ 2000	Agile	N-Tier	Virtual Servers	Hosted
				
~ 2010	DevOps	Microservices	Containers	Cloud
				

Cloud Native App -> Agile + DevOps + Microservices + Containers + Cloud

<https://www.oracle.com/in/cloud/cloud-native/what-is-cloud-native/>



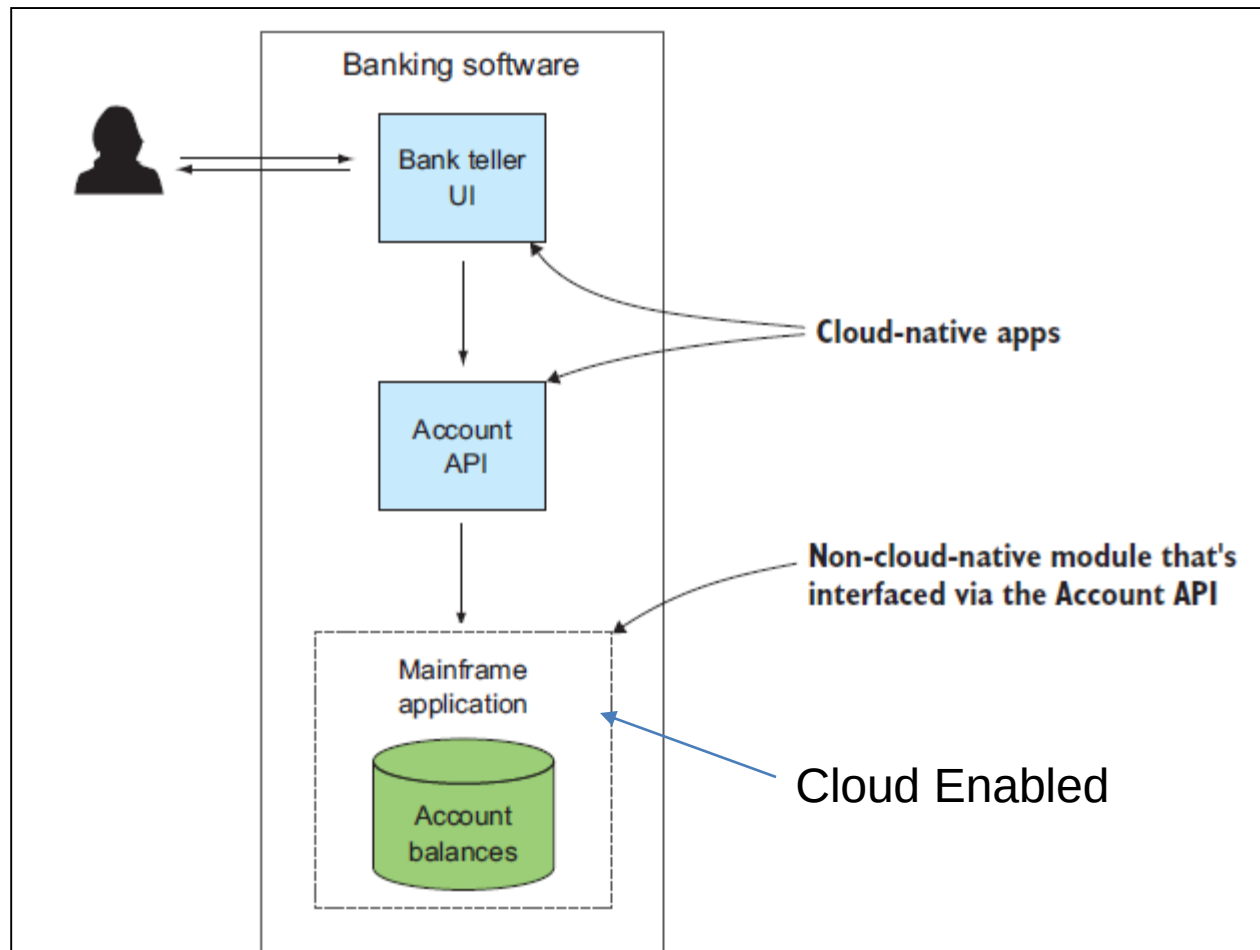
Cloud-enabled vs Cloud-based vs Cloud-native apps

Evolution of Cloud Native apps

Cloud Enabled

- Legacy applications
- Applications were built traditionally in a monolithic fashion
- They depend on local resources and hardware (on-premises)
- Integrate with services hosted on the cloud
- The application cannot take the advantage of factors like scalability as the underlying architecture remains monolithic

Cloud Enabled



Cloud Based

- Applications moved to cloud to leverage capabilities of cloud
 - Scalability
 - Higher Availability
- No need of redesign of applications to migrate to cloud
- Ex: In-house web application moved to AWS or Azure
- Do not have to worry about
 - Management of resources
 - Maintaining the servers
 - Backup
- Advantages
 - Pay for what is used
 - Scaling up/down
 - Zero downtime

Cloud Native

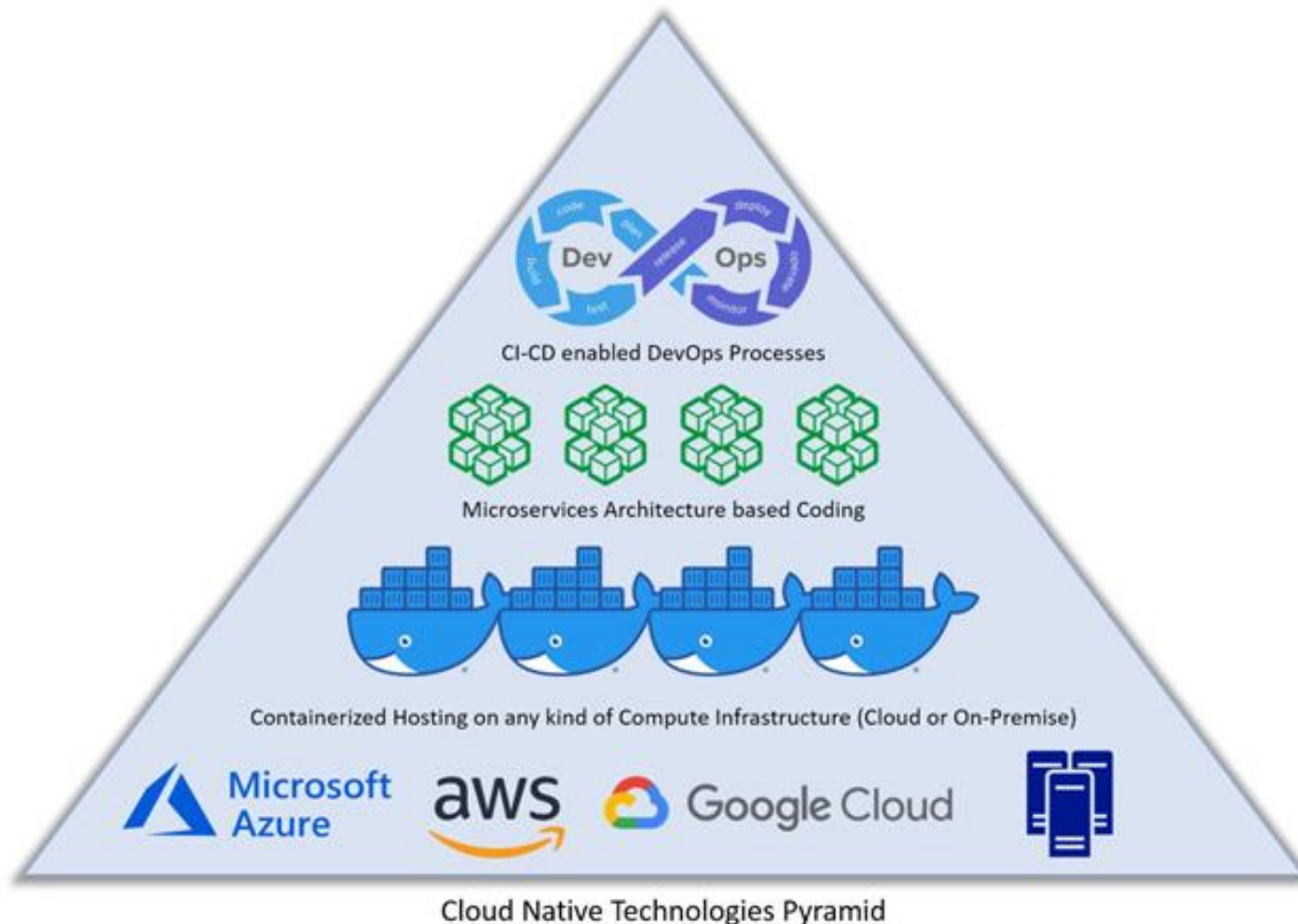
- Redhat - “Cloud-native applications are a collection of small, independent, and loosely coupled services.”
- VMWare - “Cloud native is an approach to building and running applications that exploits the advantages of the cloud computing delivery model.”
- IBM - “Cloud native refers less to where an application resides and more to how it is built and deployed.”
- Cloud-native is born in the cloud
- Cloud is about *where* we are computing; Cloud-native is about *how*



Cloud Native

- Cloud Native applications are:
 - Architected to run in a Cloud environment (Public clouds - AWS, GCP, Azure)
 - Built using cloud based technologies
 - Accessible and Scalable
- Allow developers to continuously deliver new services more quickly and easily
- Cloud-native technologies
 - Continuous integration
 - Orchestrators
 - Container engines

Cloud Native Technologies Pyramid



<https://www.ridge.co/blog/cloud-native-applications-explained/>

Cloud Native – Focus Areas

- **Design**

- **Cloud-native:** Must think about failure. Use of microservices architecture.
- **Cloud-based:** Was designed for availability.

- **Implementation**

- **Cloud-native:** Faster to deploy because there is no hardware or software to deploy.(Container Images)
- **Cloud-based:** Slower because of hardware provisioning or software setup.

- **Pricing**

- **Cloud-native:** Consumption-based-pricing (pay for what you use)
- **Cloud-based:** More expensive because you have to own the whole stack (compute – EC2, storage, monitoring & logging services) etc.

Examples of Cloud Native Applications



Netflix is a famous streaming app for TV shows, movies, and documentaries. In 2016, the organization decided to go cloud-native and shifted to microservices.



Uber is another popular app that uses a cloud-native approach. Uber has over 4,000 independent microservices, which the team monitors using the Prometheus platform. This allows Uber developers to quickly respond to market changes and update or scale a specific portion of the app.



Airbnb is a well-known American vacation rental company that operates in 65,000 cities around the world. As the company grew, it shifted to a microservices model, and as of 2022, Airbnb deploys 3,500 microservices per week, which allows the organization to keep providing services seamlessly to thousands of customers.



CNCF - Cloud Native Computing Foundation

- CNCF is the open source, vendor-neutral hub for cloud native computing
- Part of Linux foundation
- URL - <https://www.cncf.io/>
- 24 Graduated projects (stable in production environment)
 - ArgoCD, Kubernetes, envoy service proxy, Helm, Prometheus etc.
- 37 Incubating projects
 - gRPC, Thanos (monitoring), OpenTelemetry (tracing) etc.



Case Study

Importance of Application Architecture



Flipkart Big Billion Dollar Day Sale

- Launched on 6-Oct-2014
- Opened at 8am
- Big discount in 70+ categories, flash sales, lucky draw
- Sold large units of Nokia Lumia 525, Samsung Galaxy Tabs at throw-away prices
- Three Lakh order in 6 hours!

Flipkart Big Billion Dollar Day Sale

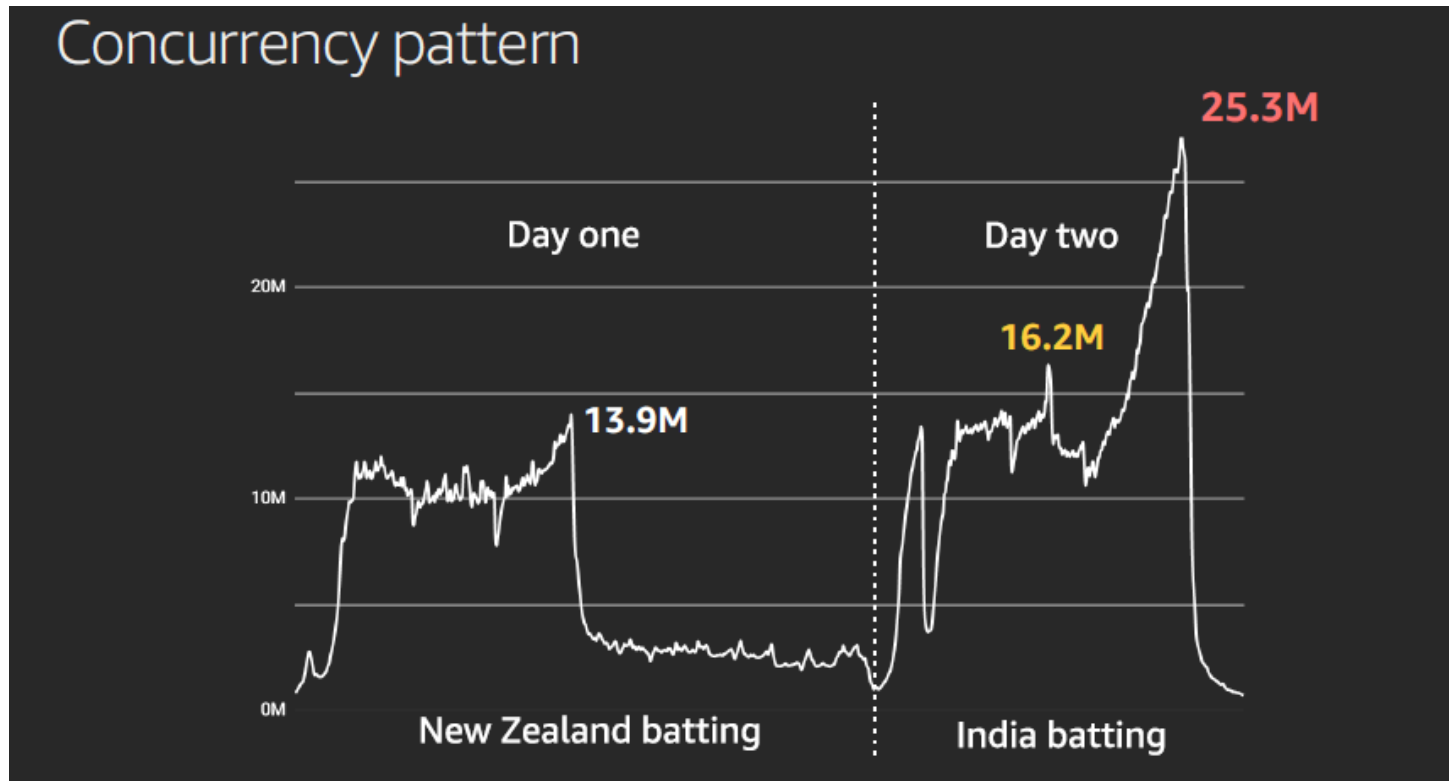


Negatives:

- Website crashed due to huge traffic and footfalls
- Already selected products vanished from cart after recovery or appeared as sold out!
- Money got deducted from account, but order not executed
- Big complaints from customers
- Reviews were hidden; no refund and no cancellation of orders

From an Architecture perspective, what could have gone wrong?

Hotstar Case Study



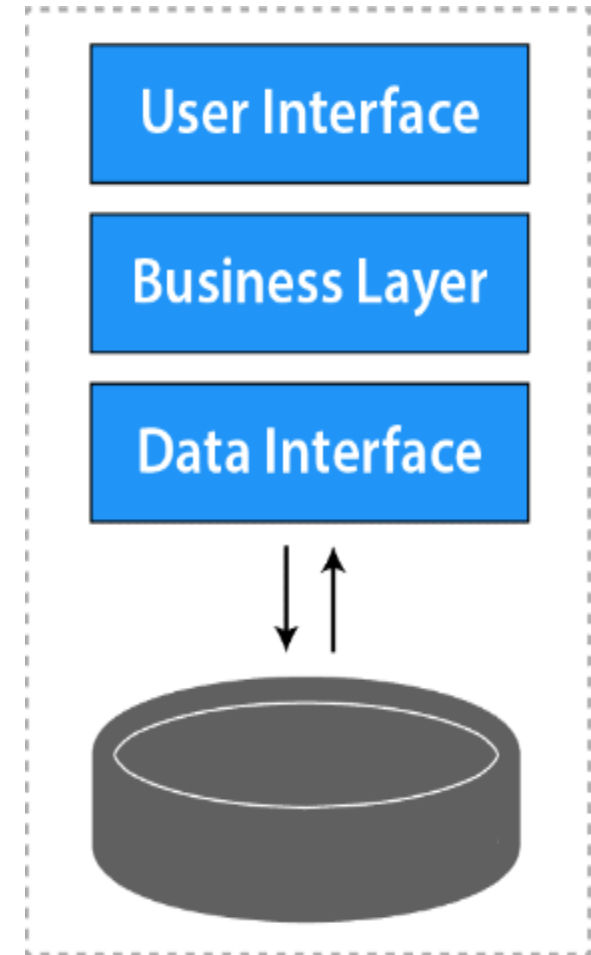
What steps did the Hotstar Cloud Architects take to handle 25.3 Million concurrent users?



Microservices

What is Monolithic Architecture?

- Monolith means composed all in one piece.
- They're typically complex applications that encompass several tightly coupled functions.
- When all functionality in a system had to be deployed together, we consider it a monolith.



Monolith Application (Food Delivery)

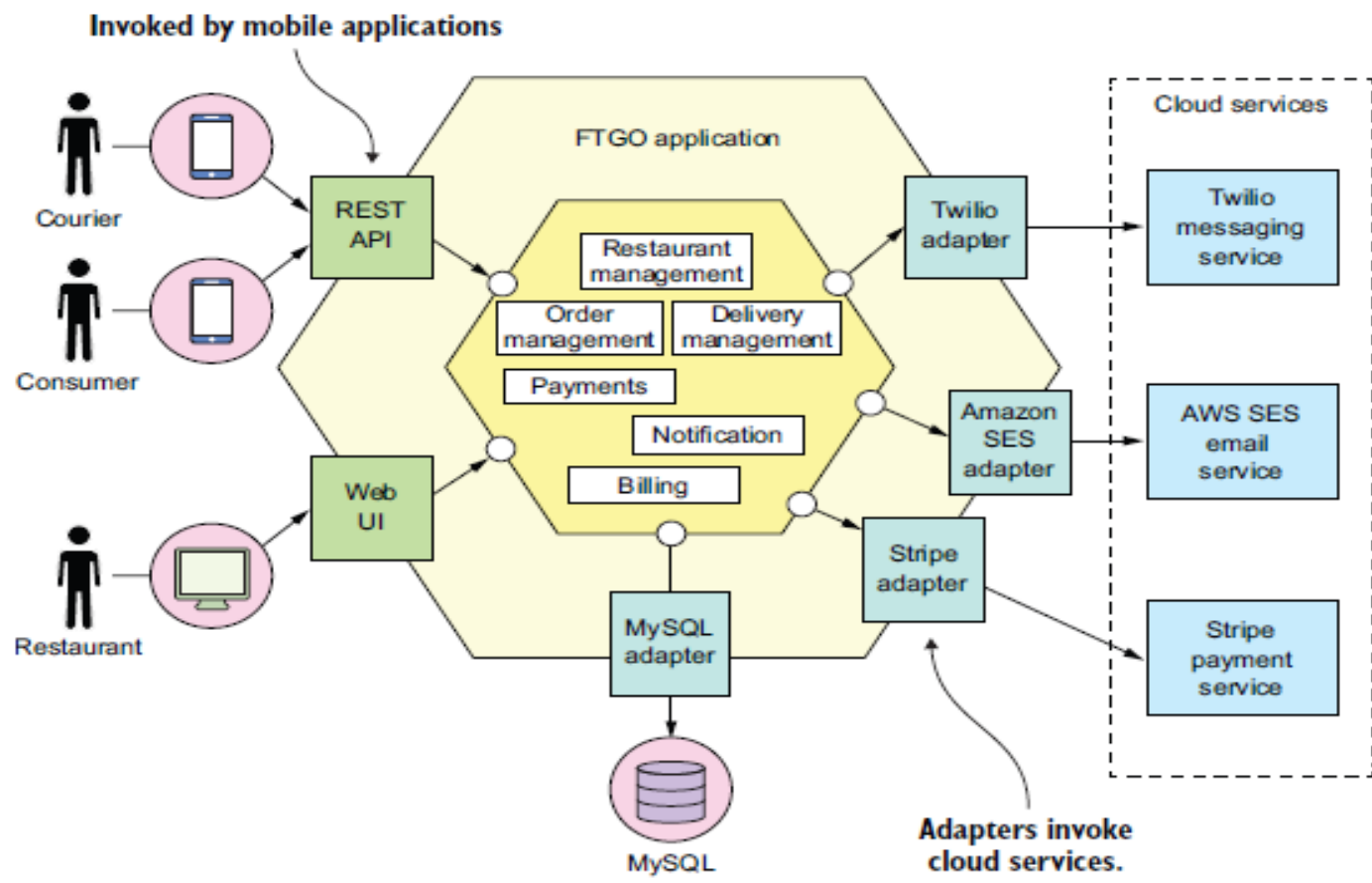
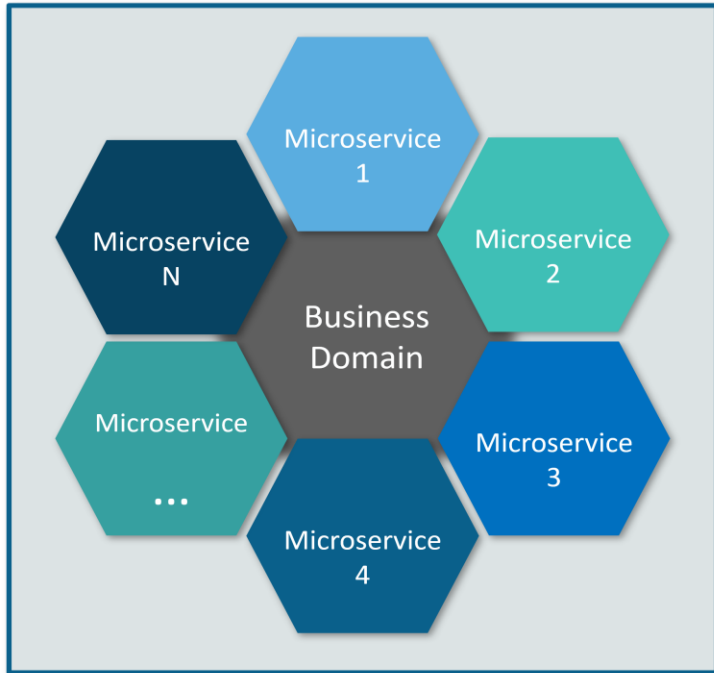


Figure 1.1 The FTGO application has a hexagonal architecture. It consists of business logic surrounded by adapters that implement UIs and interface with external systems, such as mobile applications and cloud services for payments, messaging, and email.

Monolithic Limitations

- *Technology Barrier* - embracing new technologies means application needs to be re-written
- *Scalability* - The only option is scaling the whole application, since we can't scale the components independently
- *Size* - As application size increases, complexity increases, may become unmanageable
- *Difficult to understand* - Every new joiner needs to understand the entire application, not just his/her modules, because of dependencies [Maintenance project problems]

What are Microservices

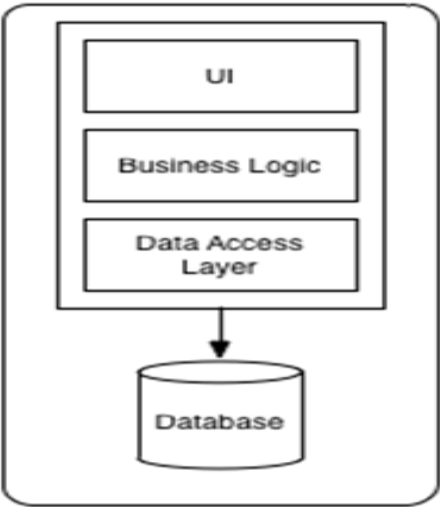


- An architectural style
- Microservices are autonomous, loosely coupled, and independently deployable services modeled around a business domain
- Each microservice can focus on a single business capability

Robert C. Martin's Single Responsibility Principle

*Gather together the things that change for the same reasons.
Separate those things that change for different reasons.*

Why Microservices are required

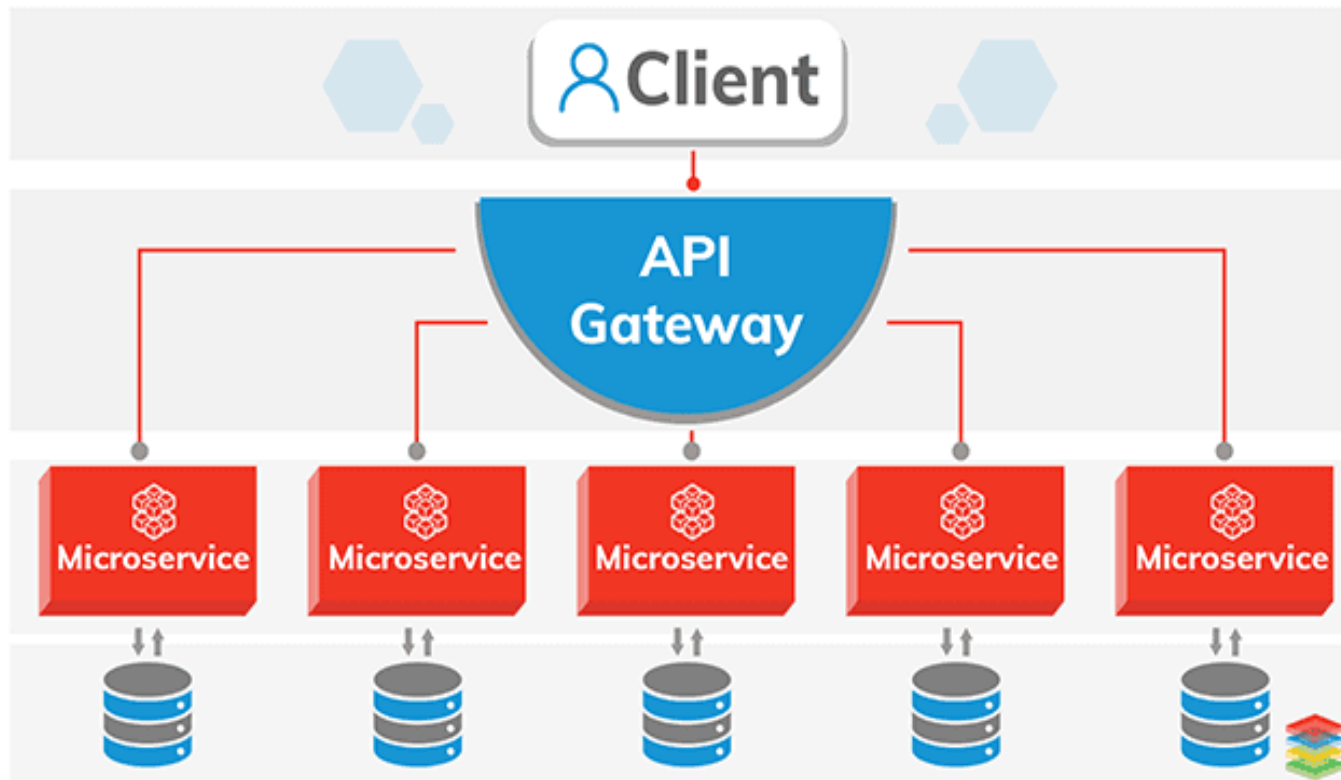


Monolithic Architecture

Company	Deployments
Amazon	23,000/ day
Google	5,500 / day
Netflix	500 / day
Twitter	3 / week

Use Case - Shopping Cart Application (Amazon, Flipkart etc.)
Problems with implementing Shopping Cart as Monolithic Architecture
Scenario 1: Agility
Immediate changes to the application needs entire code to be rebuilt for every small change (version management).
Scenario 2: Hybrid Technologies
Developers are from varying skillsets such as C#, Java, Python etc. They are forced to build the shopping cart application using a single technology.
Scenario 3: Fault Tolerance
A feature (Ex: cart amount debit) is not working in the application. A fix implies that the entire application has to be re-built, re-tested and re-deployed.
Scenario 4: Scalability
Developers cannot scale the application simultaneously. New instances of the same application have to be created every time a new feature has to be developed or deployed.

Example Architecture

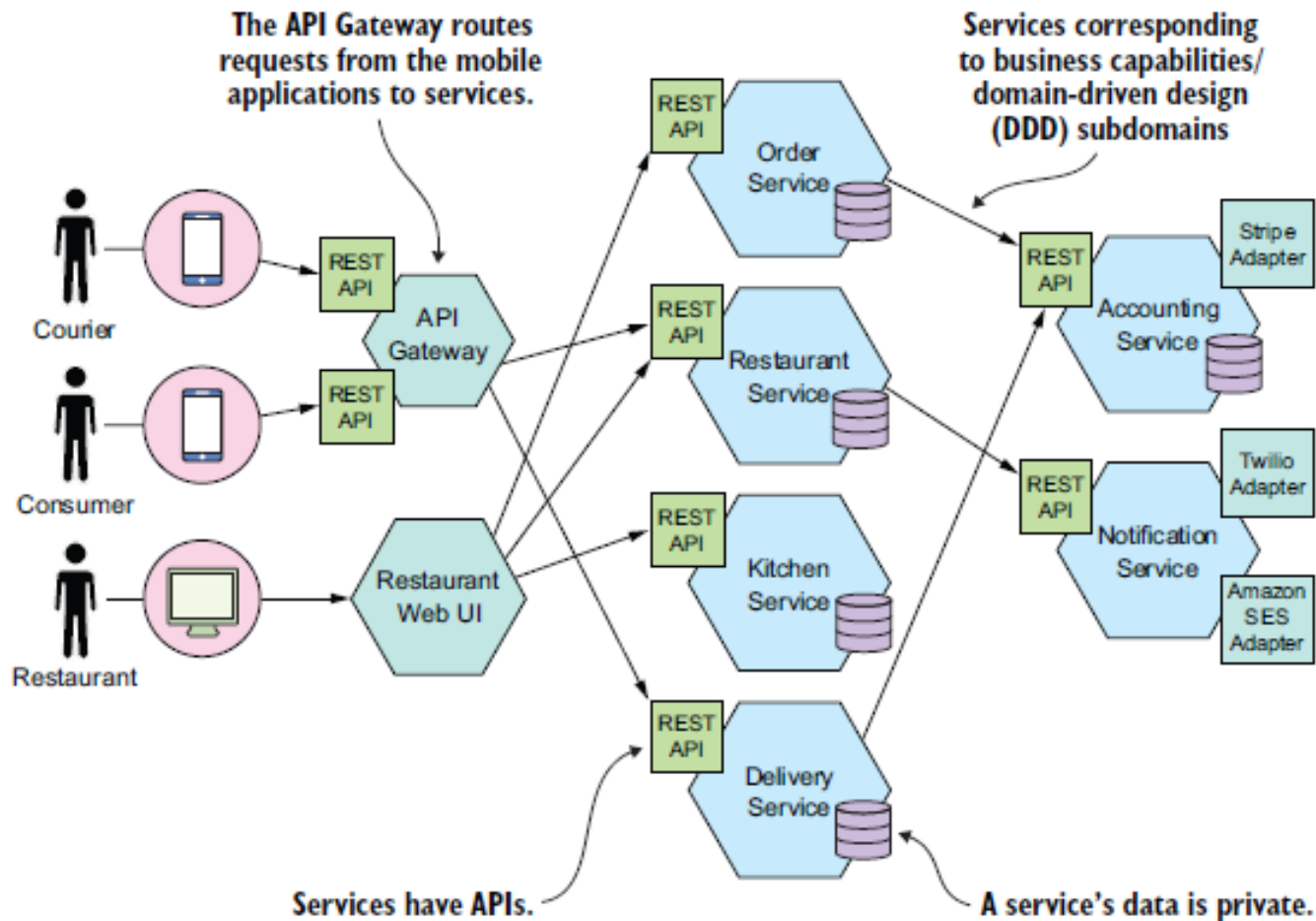


Frequency	Percentage
Never	10%
Sometimes	40%
Often	50%



Microservices Architecture (Food Delivery)

API-First Design



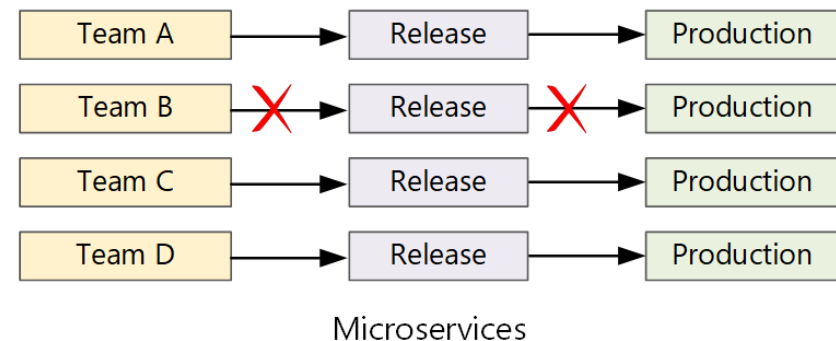
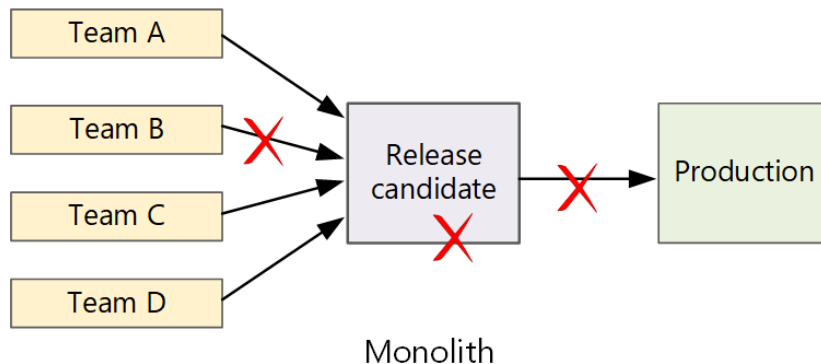
CI/CD pipeline for Monolith vs Microservices

Monolith

- Single Build Pipeline
- Bug fixing delays release of features

Microservice

- One service, one pipeline [pipeline-per-service pattern]
- High release velocity and reliability



<https://learn.microsoft.com/en-us/azure/architecture/microservices/ci-cd>

FTGO - Monolithic Hell - Pipeline

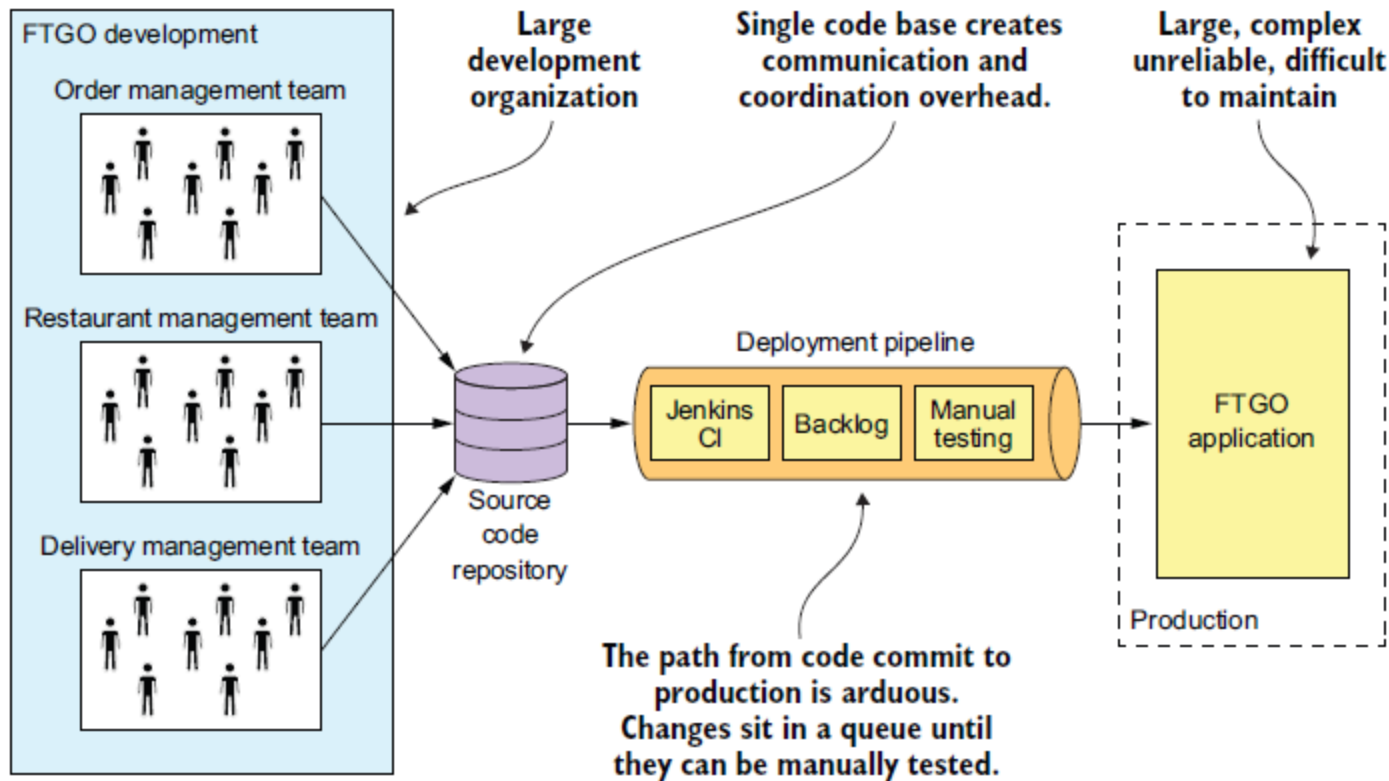
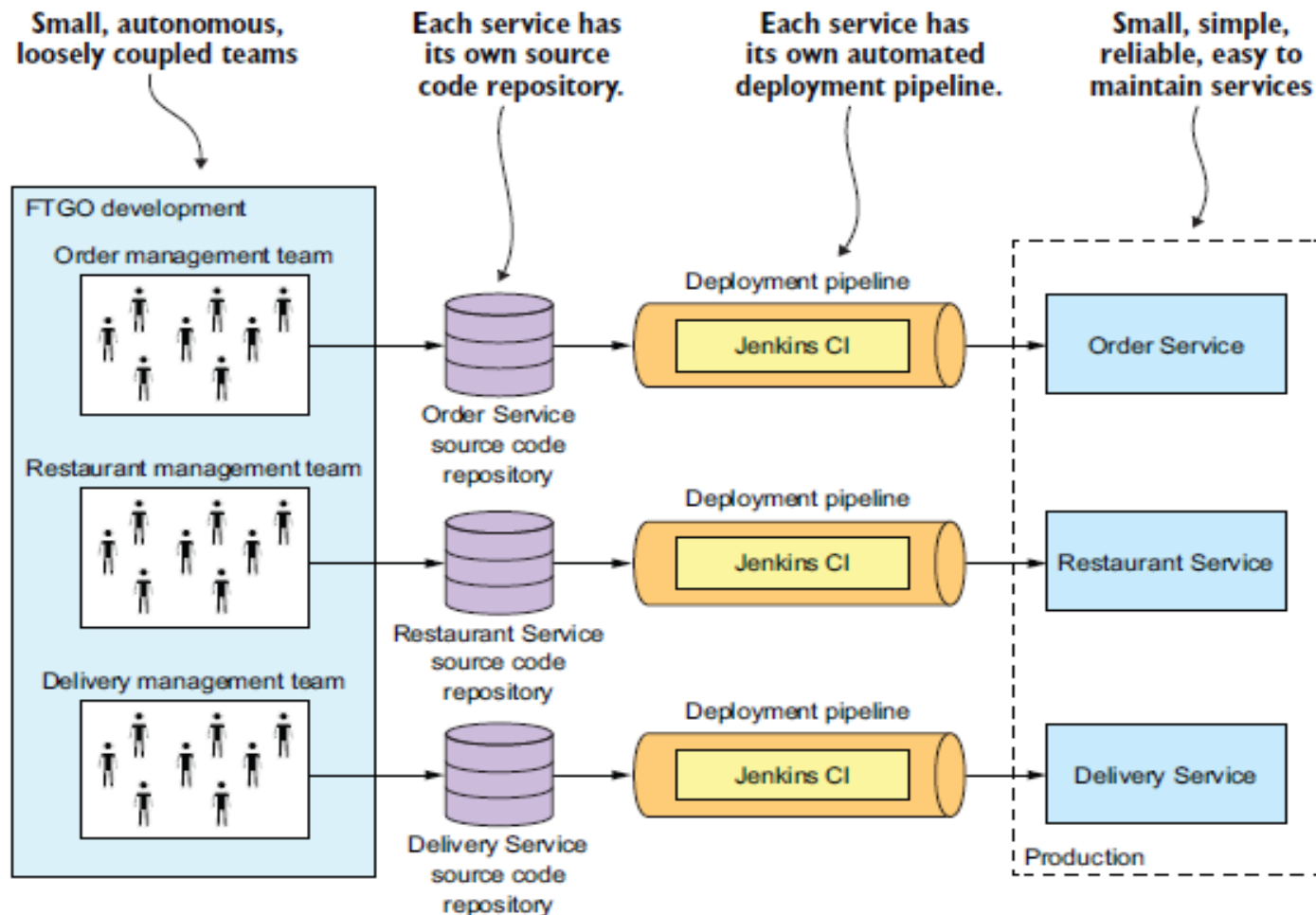


Figure 1.2 A case of monolithic hell. The large FTGO developer team commits their changes to a single source code repository. The path from code commit to production is long and arduous and involves manual testing. The FTGO application is large, complex, unreliable, and difficult to maintain.

Microservices Pipeline – FTGO Application





Recent shift back towards Monolith

- Amazon Prime Video Streaming Service has ditched its use of “microservices-cum-serverless” model, reverting to a traditional, monolithic architecture.
- It vastly improved the workload’s **cost and scalability**.
- The team moved all the components into a single process, hosting them on...EC2 and...ECS.
- Achieved a 90% reduction in operational costs

Ref

1. Article - <https://devops.com/microservices-amazon-monolithic-richixbw/>
2. Architecture - <https://nordicapis.com/back-to-the-monolith-why-did-amazon-dump-microservices/>



BITS Pilani
Pilani Campus



Serverless Computing

Some MCQs on Serverless

1. What is the main advantage of serverless computing?
 - A) Reduced manual server management
 - B) Enhanced data security
 - C) Increased physical infrastructure
 - D) Enhanced UI/UX design

2. Which of the following is NOT a component typically required to complete a serverless stack on AWS?
 - A) Compute service
 - B) Database service
 - C) Virtual Machine service
 - D) HTTP Gateway service

3. What is the main role of AWS Lambda in a serverless stack?
 - A) It serves as a database service
 - B) It acts as the compute service
 - C) It manages HTTP requests
 - D) It handles networking

Some MCQs on Serverless

4. Which of the following is considered a drawback of serverless deployment?

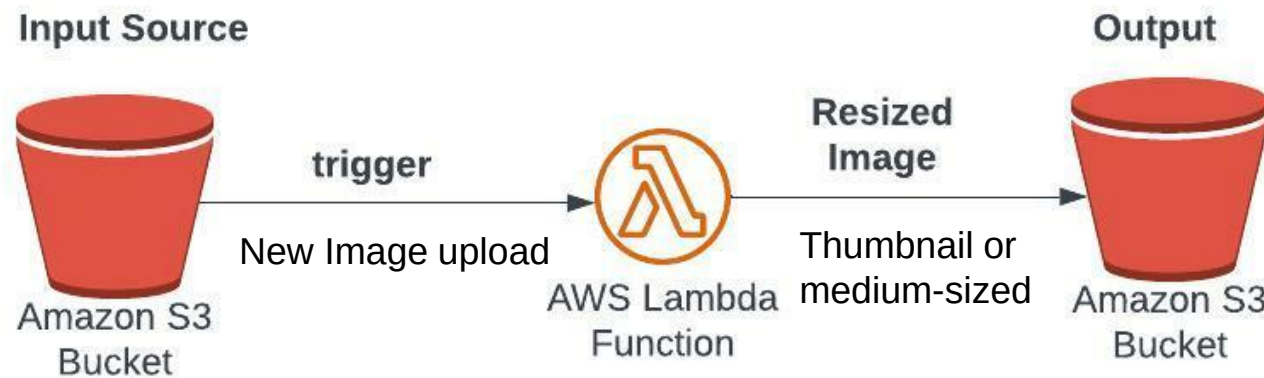
- A) High cost of operation
- B) Vendor lock-in
- C) Difficulty in scaling
- D) Complex manual configuration

5. What happens during a 'cold start' in AWS Lambda?

- A) The function runs instantly without any delay
- B) A new container is created because there is no existing warm container
- C) The function fails to execute
- D) The function is triggered multiple times

Serverless Computing

- Application deployment paradigm that allows applications to run on-demand, consuming only the resources required to execute them
- Build and run applications and services without managing infrastructure
- Developers can focus on their core product
- Ex: AWS Lambda, Google Cloud Functions (GCF), Azure Functions are examples of FaaS (Function as a Service)



Serverless Computing

Serverless help developers focus on building, not managing

Focus on creating
business value

CUSTOMER



AWS

Remove heavy lifting with
serverless everything

AWS Serverless Services

Serverless is much more than compute

COMPUTE



AWS
Lambda



AWS
Fargate

DATA STORES



Amazon
S3



Amazon
Aurora Serverless



Amazon
DynamoDB

INTEGRATION



Amazon
EventBridge



Amazon
API Gateway



Amazon
SQS



Amazon
SNS



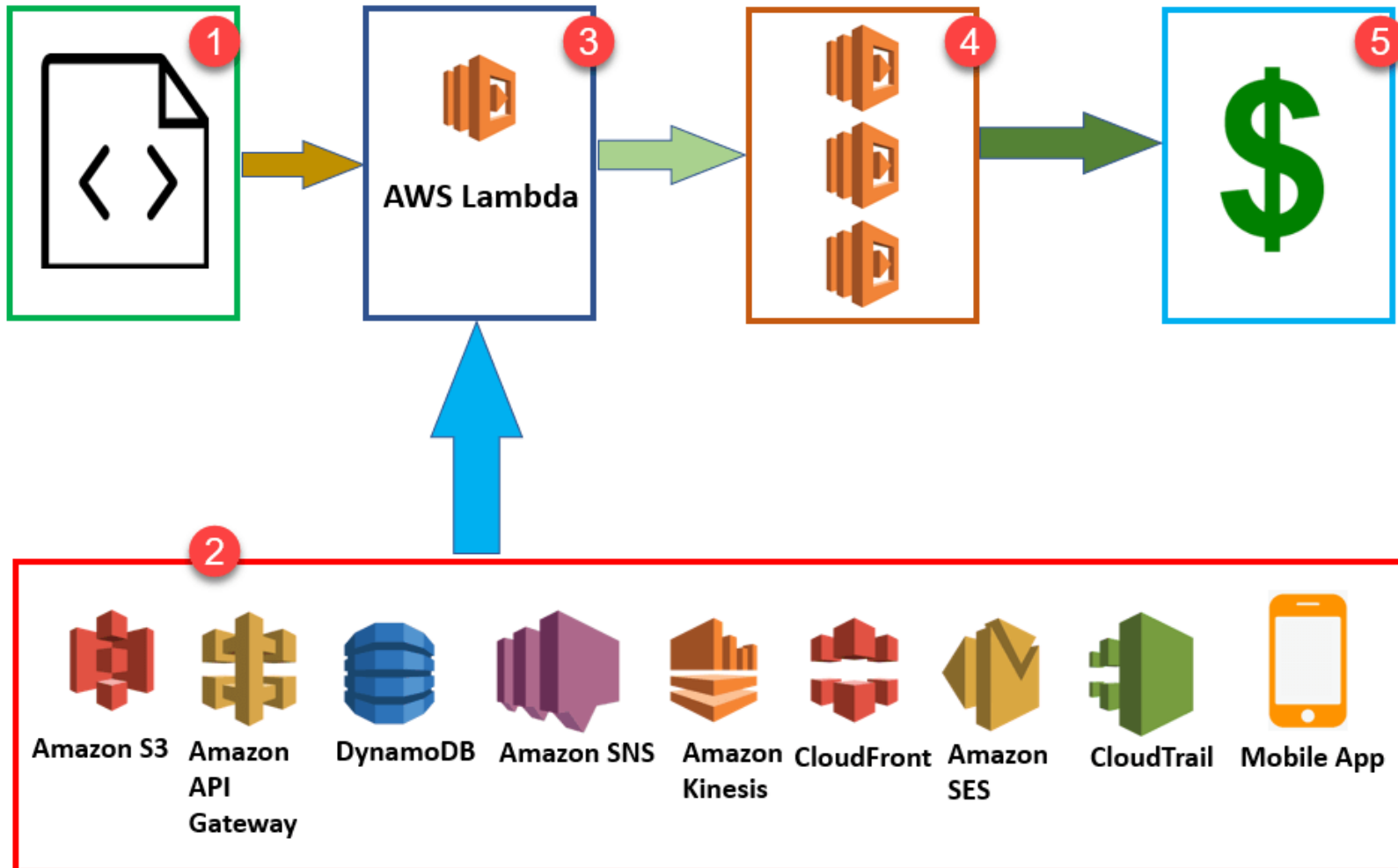
AWS Step
Functions



AWS
AppSync

AWS Lambda

(FaaS - Function as a Service)

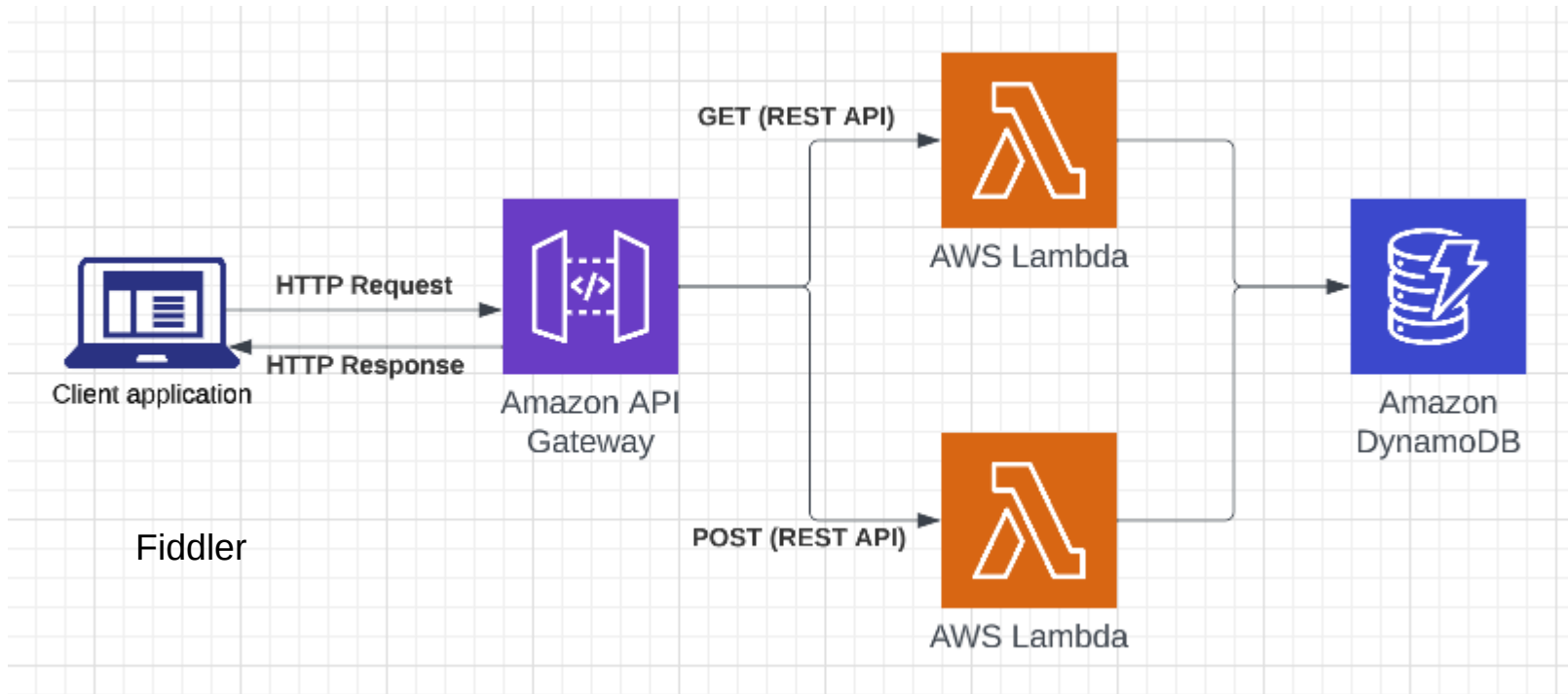


Serverless Stack

- When building Serverless applications, AWS Lambda is one of the main candidates for running the application code. Typically, to complete a Serverless stack you'll need:
 - A Compute service;
 - A Database service; and
 - A HTTP Gateway service.
- Lambda fills the primary role of the compute service on AWS.
- It also integrates with many other AWS services and, together with API Gateway, DynamoDB and RDS, forms the basis for Serverless solutions for those using AWS.
- Lambda supports many of the most popular languages and runtimes, so it's a good fit for a wide range of Serverless developers.



Demo of Serverless Application



- AWS Lambda (Get and Post functions) are Serverless Compute
- Amazon DynamoDB is the Serverless Database
- Amazon API Gateway is the HTTP Gateway Service

Drawbacks of Serverless Deployment

- It's not the best scenario for executing long-running applications [No control over the environment]
- Vendor lock-in
- Cold start [AWS Lambda typically keeps containers alive for 30-45 minutes. For a new request, if a function is not running in a warmed container, a new container will be created called 'cold start'. Latency (more time) to execute the request]



BITS Pilani
Pilani Campus



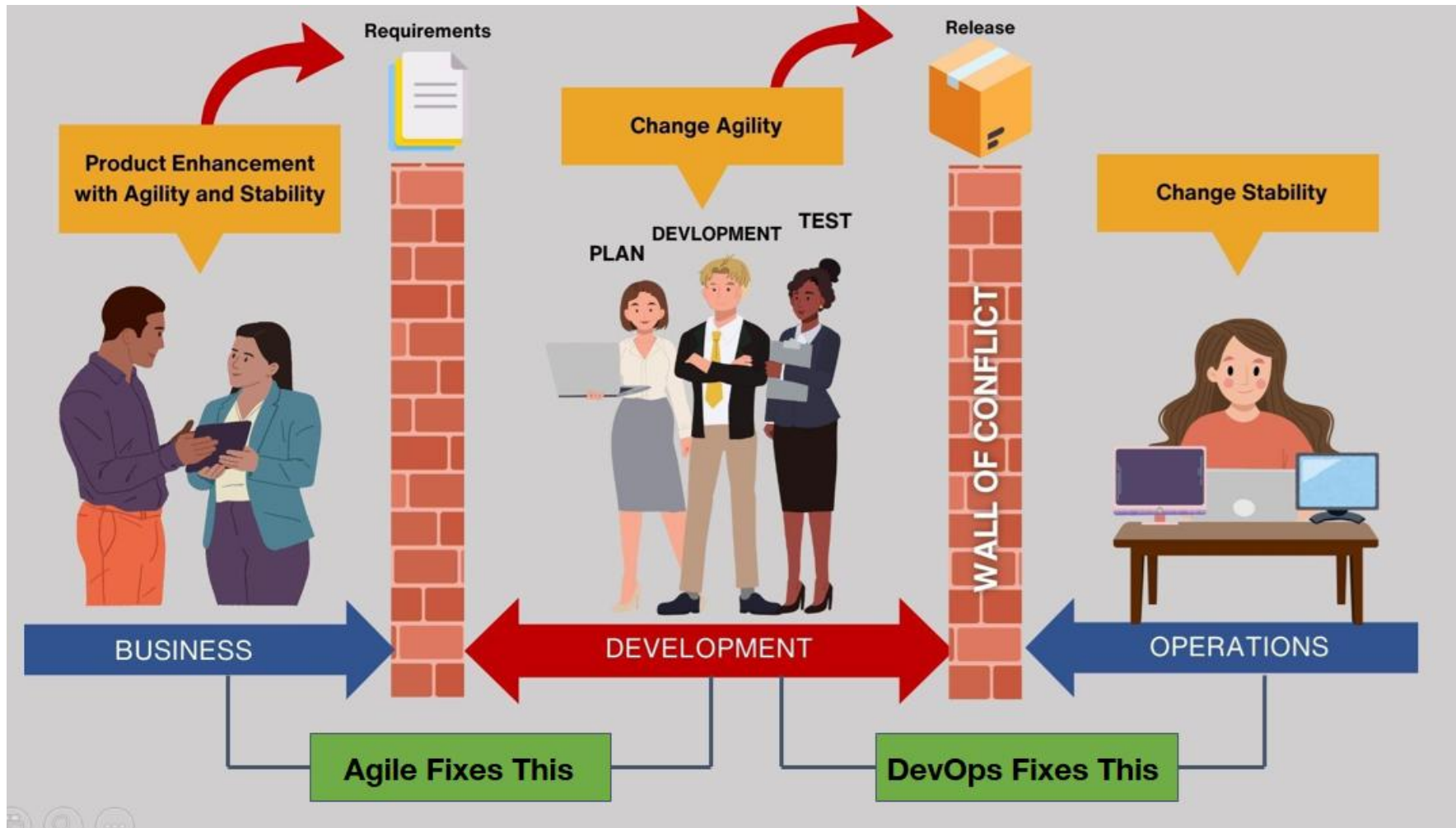
DevOps

Need for DevOps

innovate

achieve

lead



DevOps - Definition



1. DevOps is the process of alignment of IT Development and Maintenance Operations with better and improved communication.

2. Microsoft defines DevOps as “Union of People, Process, and Products to enable continuous delivery of value to the customers”

People -> Culture

Process -> DevOps Methodology

Products -> Tools

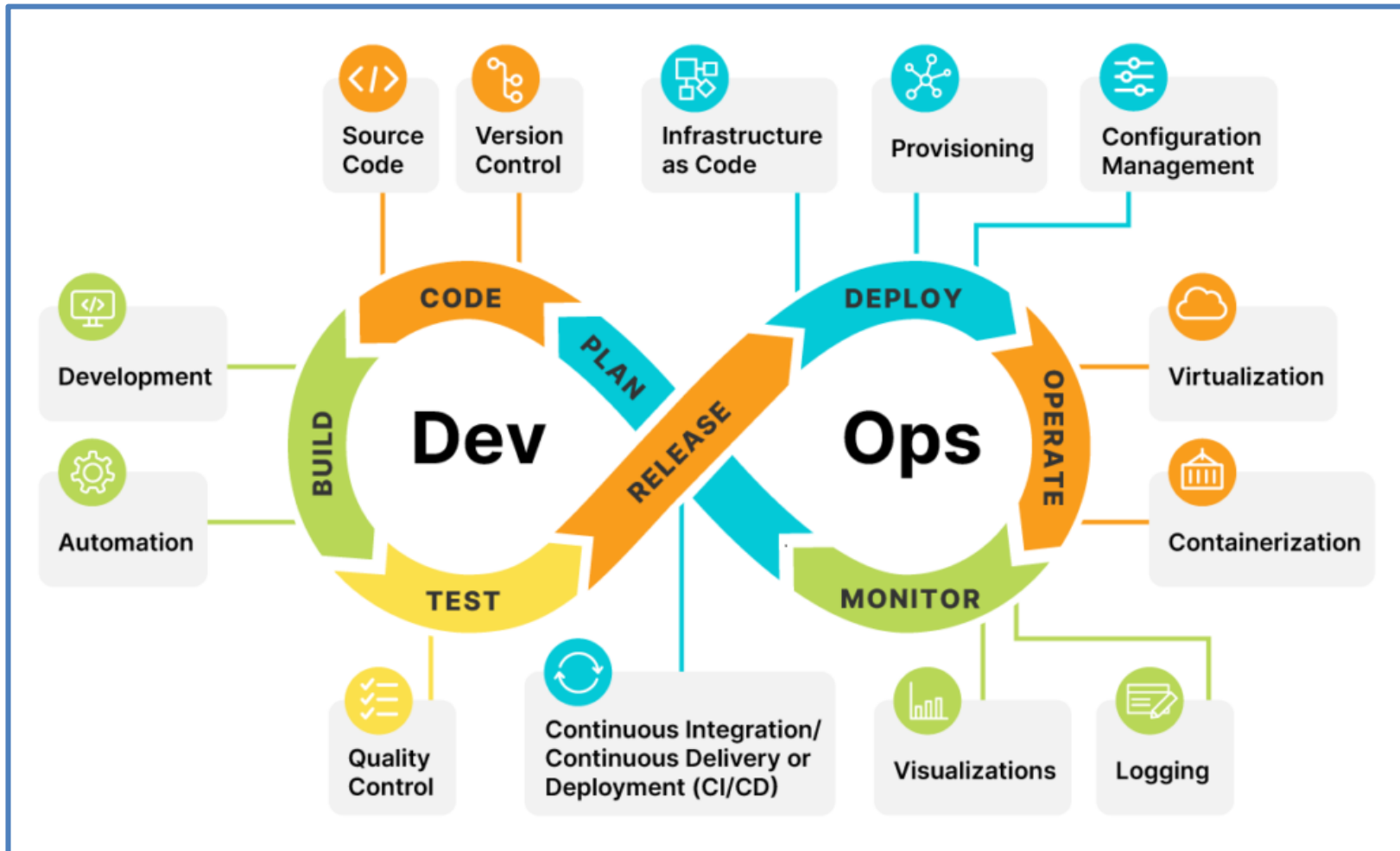
DevOps Dimensions



- ★ DevOps is a Philosophy
- ★ Dimensions enhance collaboration
- ★ Involves People, Process and Tools



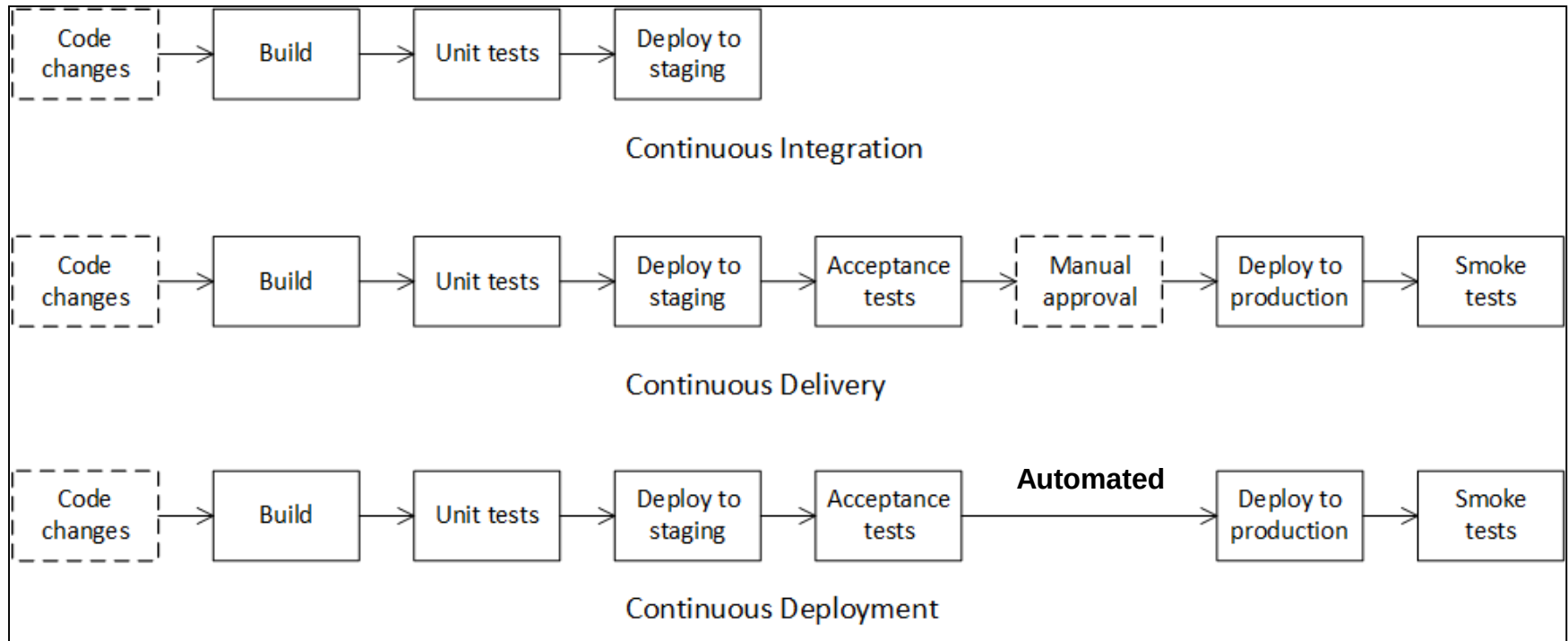
DevOps Process



CI/CD practices in DevOps

- Continuous Integration, Continuous Delivery, Continuous Deployment (CI/CD) are **DevOps practices** for producing software in short cycles between merging source code changes and updating applications.
- The ultimate goal of these practices is to:
 - Reduce the costs
 - Save time
 - Mitigate risks by delivering software in small pieces

CI/CD practices in DevOps

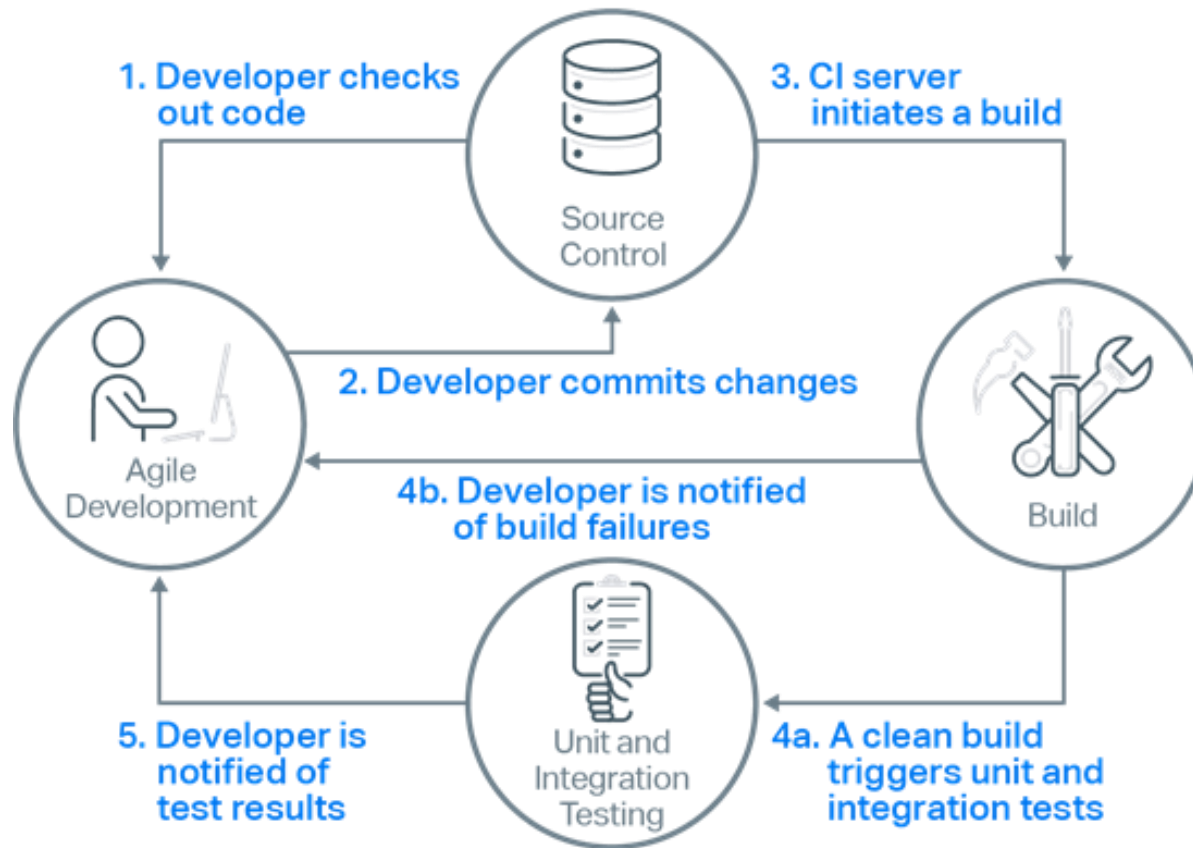




Continuous Integration (CI)

- Continuous Integration is a software development practice in which developers regularly commit and push their local changes back to the shared repository (such as GIT, usually several times a day).
- Before each commit, developers can run unit tests locally on their source code as an additional check before integrating.
- A continuous integration service automatically builds and runs unit tests on the new source code changes to catch any errors immediately.

CI Workflow



Continuous Delivery (CD)

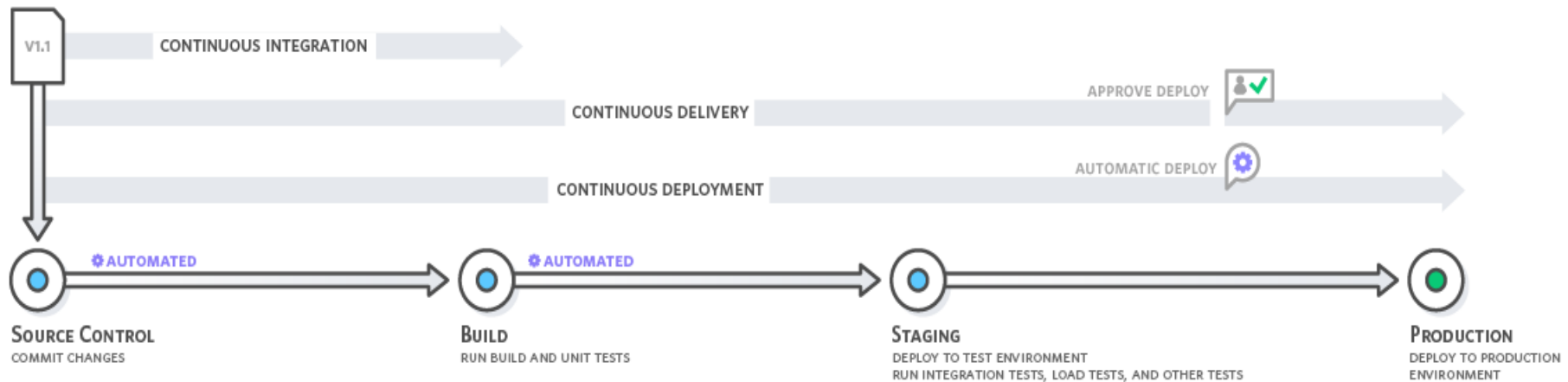
- Source code changes are automatically prepared for deployment to a production instance.
- After a build, the build artifact with new changes is deployed to a staging instance where advanced (integration, acceptance, load, end-to-end, etc.) tests are run.
- If needed, the build artifact is deployed to the production instance after **manual approval**.

Continuous Deployment (CD)

- Extends Continuous Delivery in which source code changes are **automatically deployed** to a production instance.
- The difference between Continuous Delivery and Continuous Deployment is the presence of manual approval.
- With Continuous Delivery, deployment to production occurs automatically after manual approval.
- With Continuous Deployment, deployment to production occurs automatically without manual approval.

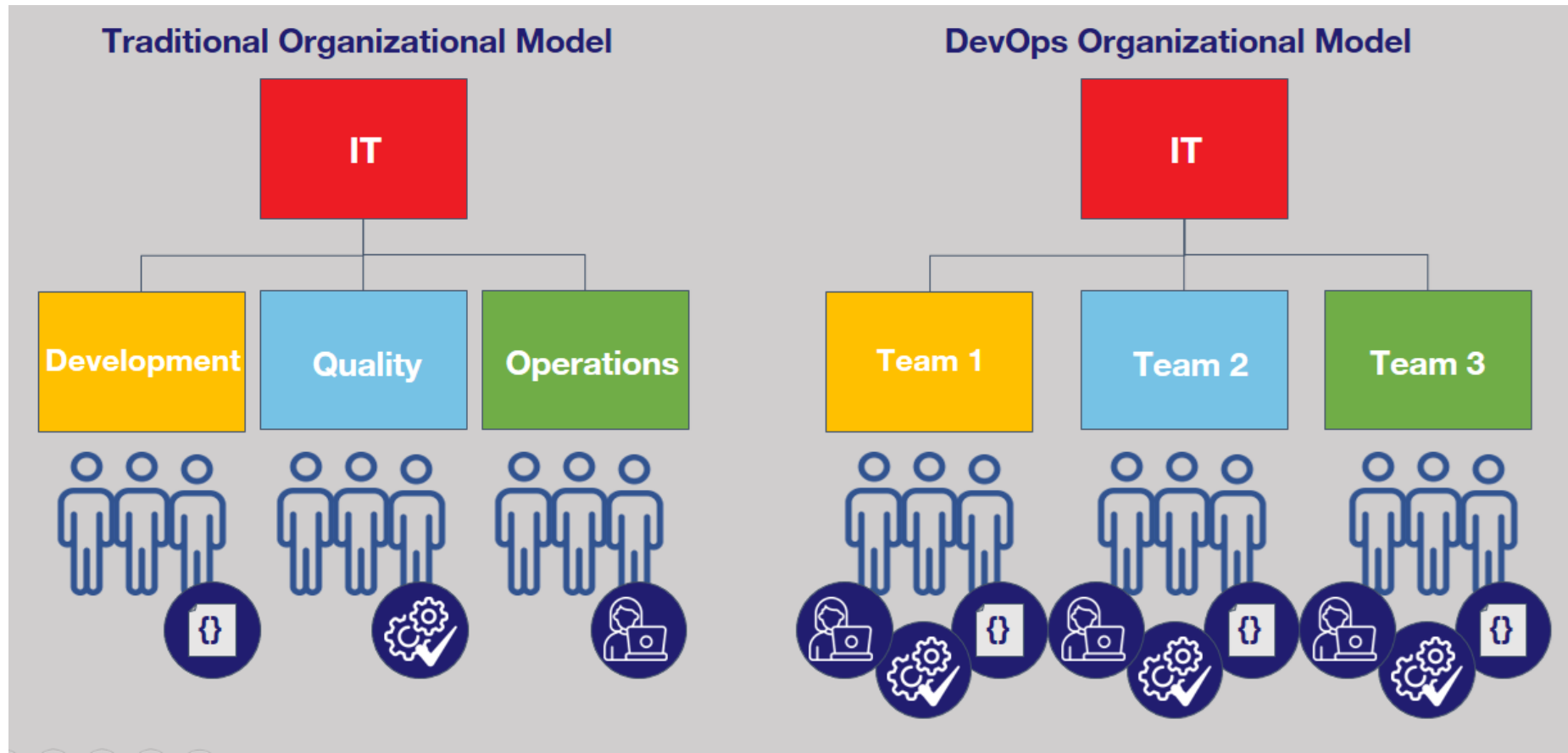
CI / CD Pipeline

A CI/CD pipeline is a series of steps that must be performed in order to deliver a new version of software



<https://aws.amazon.com/devops/continuous-integration/>

Team Formation





BITS Pilani
Pilani Campus

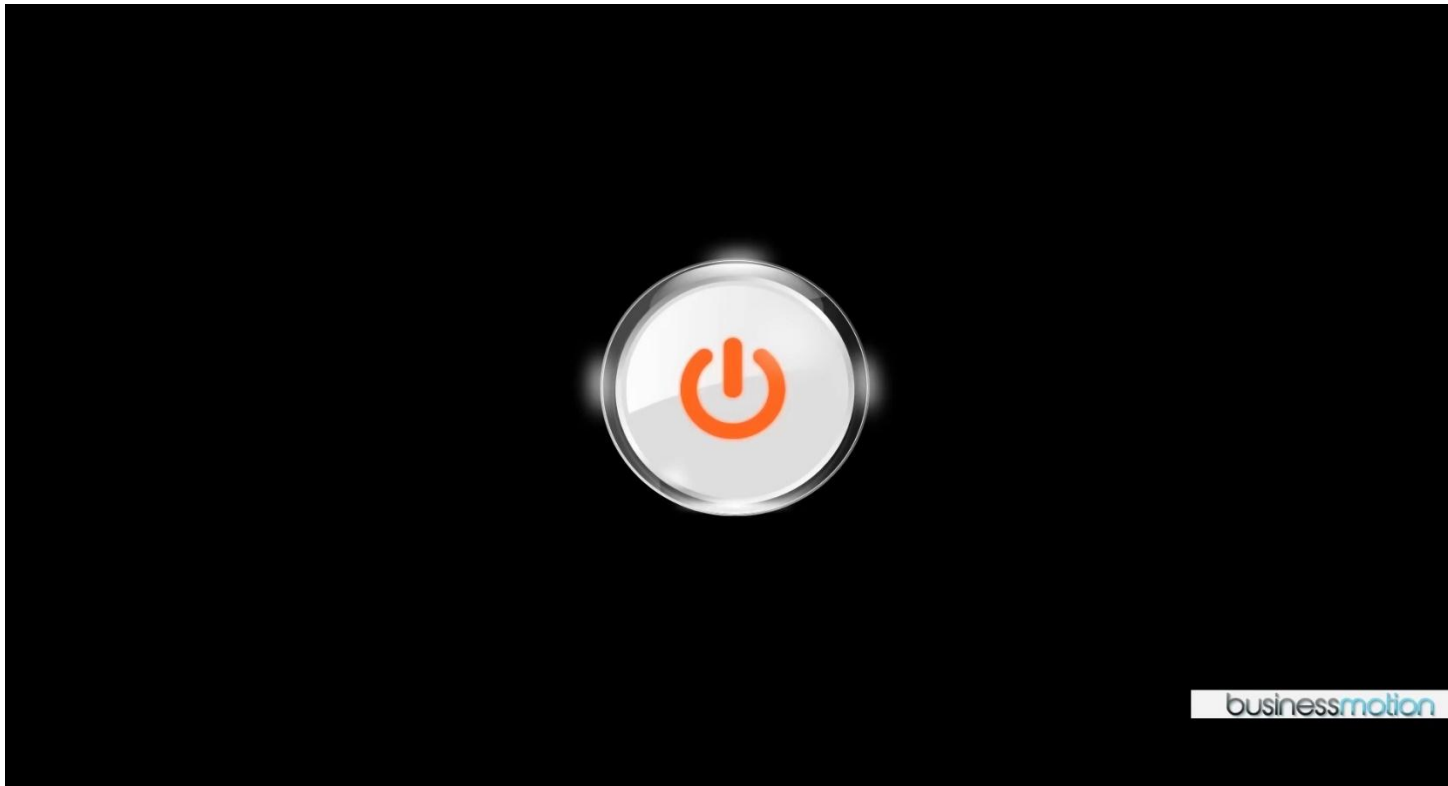


GitOps

GitOps

- Term coined in 2017 in a series of Blogs by Alexis Richardson, cofounder and CEO of Weaveworks.
- Term is used in Cloud Native Community and Kubernetes Community
- GitOps is a set of best practices and principles for *managing applications* and **infrastructure configurations** using **Git** as the single source of truth for **declarative configuration** and version control.
- Developer-centric experience for managing applications with fully automated pipelines or workflows.

What is GitOps and How it Works!



<https://www.youtube.com/watch?v=f5EpcWp0THw>

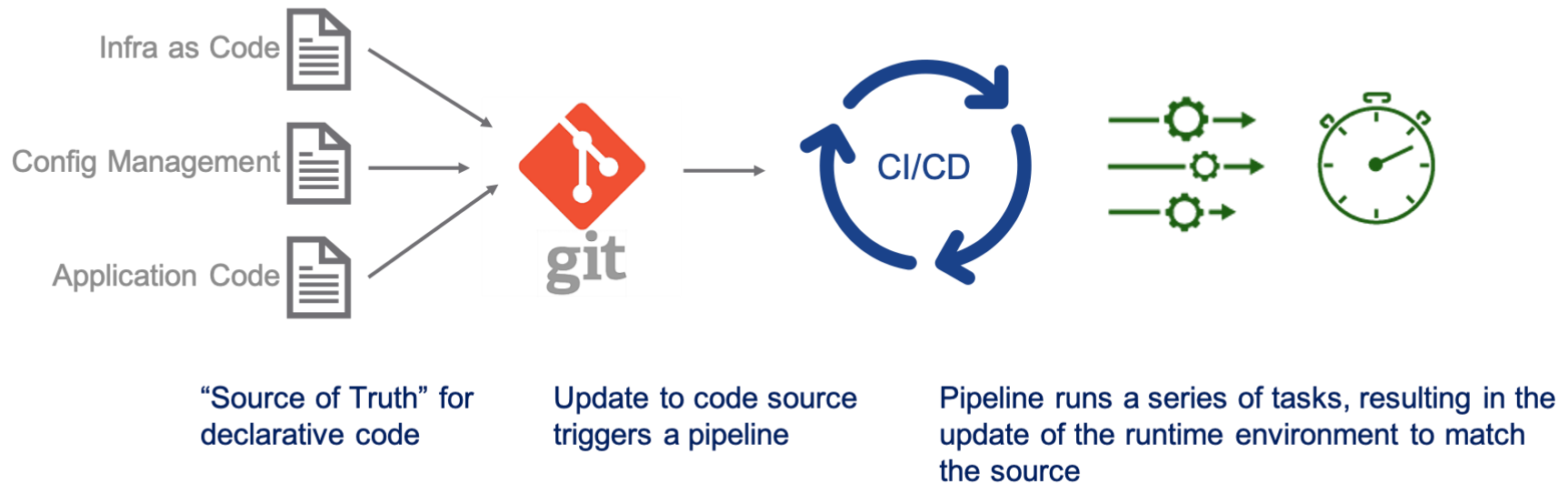
[Duration – 10 minutes]

GitOps Definitions

- Atlassian describes GitOps as “code-based infrastructure and operational procedures that rely on Git as a source control system”
- GitLab describes “an operational framework that takes DevOps best practices used for application development such as version control, collaboration, compliance, and CI/CD, and applies them to infrastructure automation”

GitOps

GitOps-in-a-nutshell



GitOps Tooling

Tooling Type	Example
Git code repository	Git
Git management tool	Bitbucket, GitHub, GitLab
Continuous integration tool	CircleCI, Jenkins, Jenkinsx, GitHub Actions
Continuous delivery tool	ArgoCD, FluxCD, Spinnaker
Container registry	Docker Hub, AWS ECR, GHCR
Infrastructure provisioning	AWS CloudFormation, Pulumi, Terraform, AWS SAM
Configuration manager	Ansible, Chef, Puppet, Helm charts (config specific to Kubernetes)
Container orchestration	Kubernetes, Nomad

Demo [Additional – if required]

Objective

1. Use GitHub Actions to define a Continuous Integration/Continuous Delivery Pipeline for the Python RESTful API Application
2. Demonstrate the Workflow with Build, Test, and Deploy as jobs in the pipeline



Containers

Docker / Kubernetes

Singer Analogy

01

A Singer is invited for a concert. He goes there and Mike does not work

02

Singer is invited to second concert, where Speakers do not work

03

Singer now invests in his own mike and speakers, and takes them to whichever concert he goes.

This way the vagaries of the environment (mike, speaker) are avoided

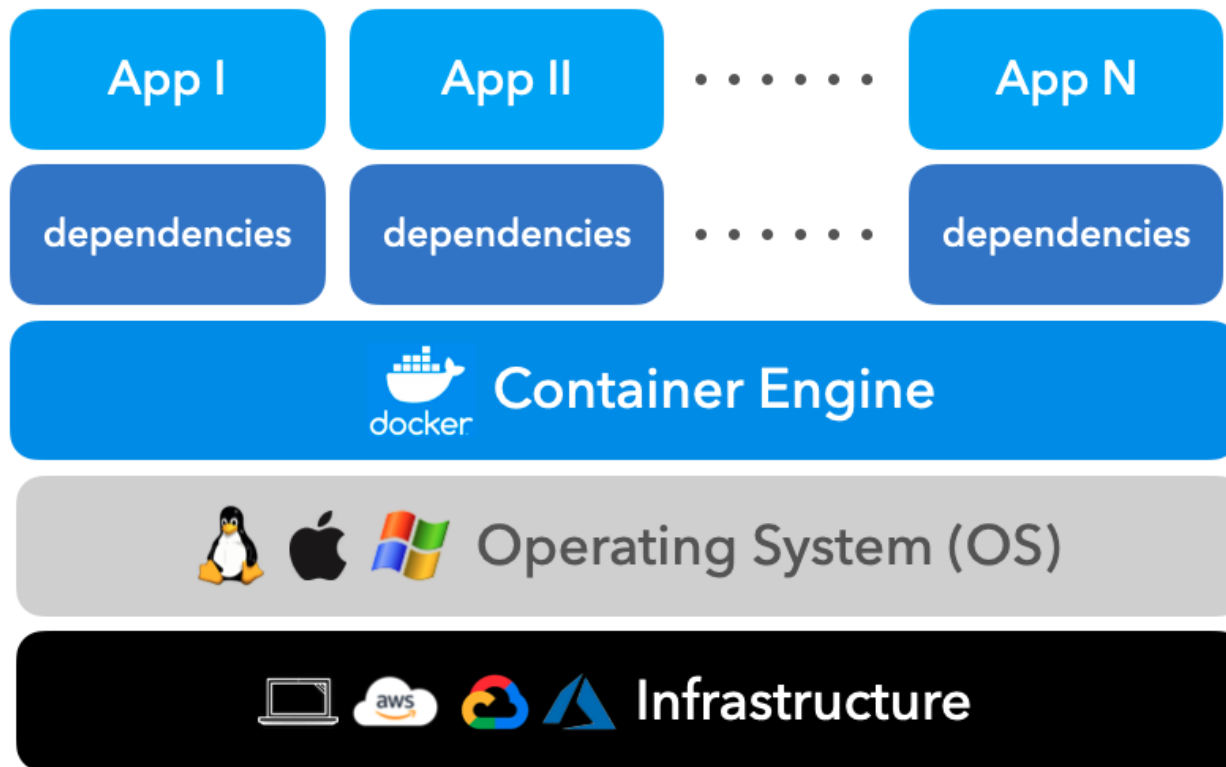
What are Containers

- A software container is a standardized package of software
- **Everything needed for the software to run is inside the container**
- The software code, runtime, system tools, system libraries, and settings are all inside a single container
- **Container based deployments is favoured for Microservices**

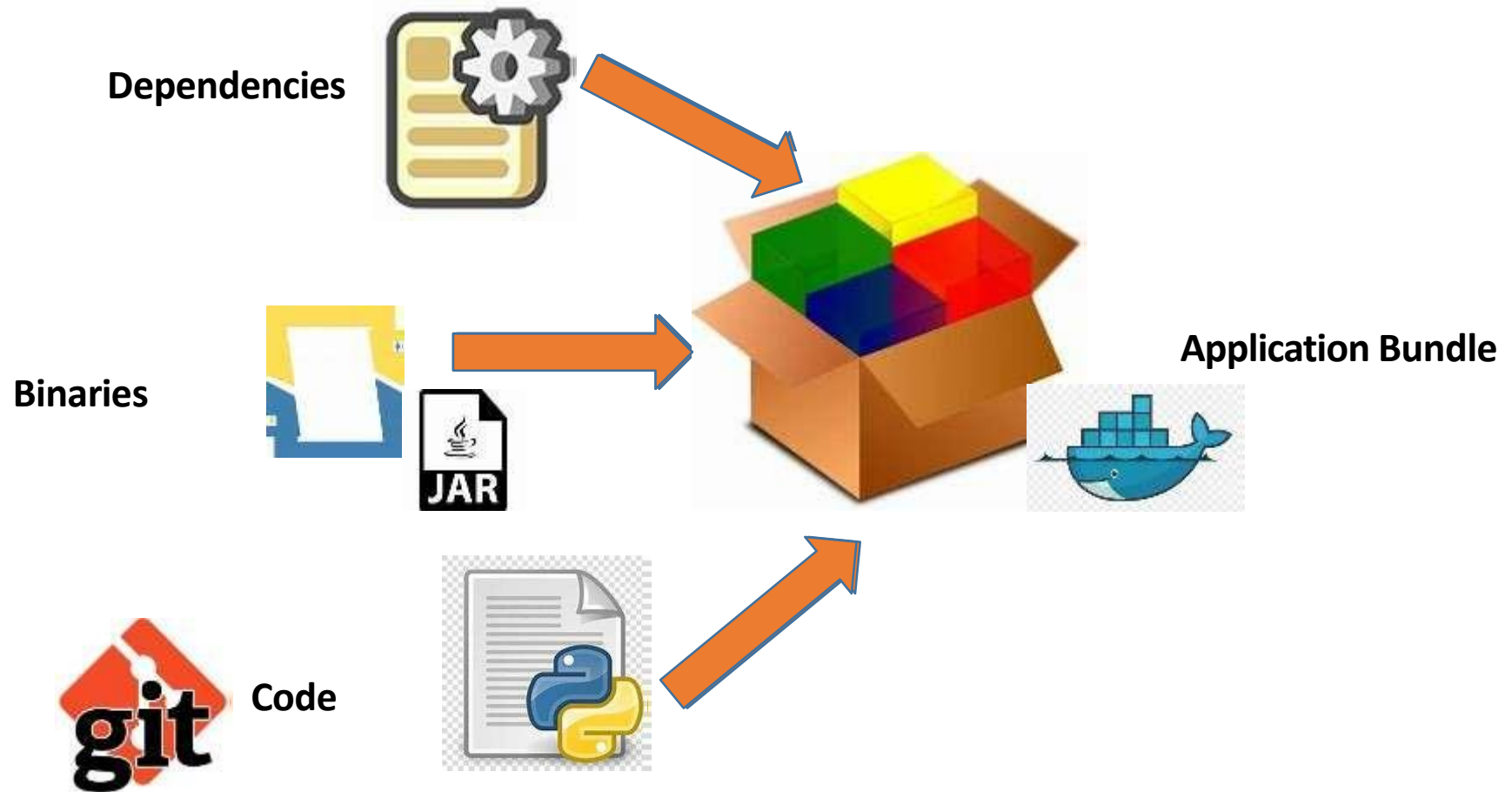


Docker

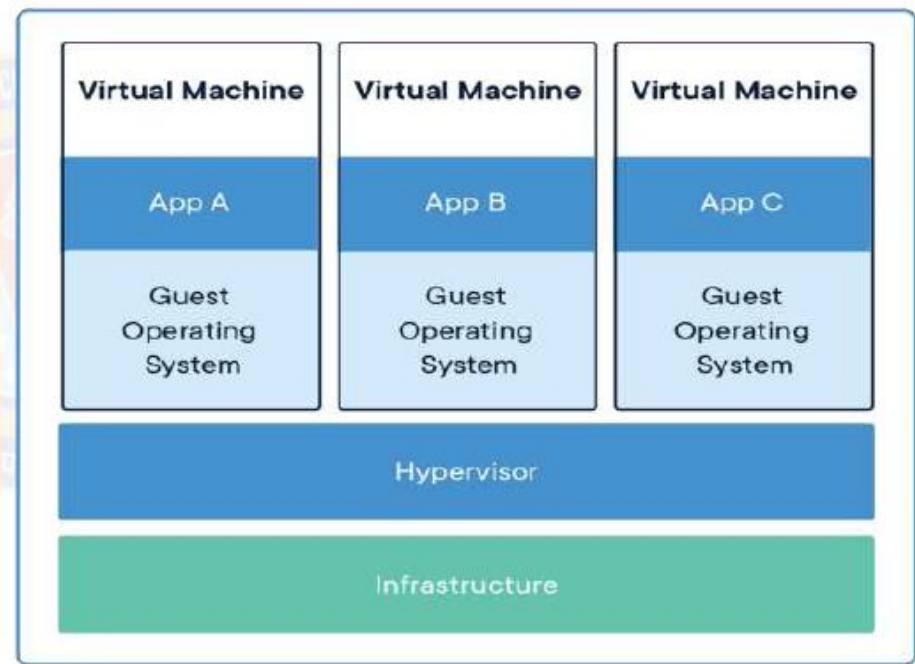
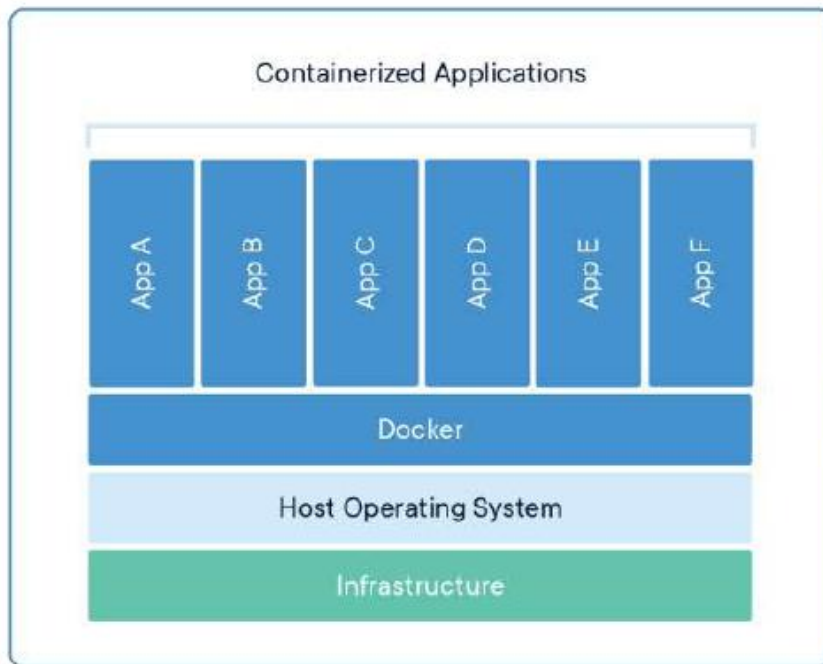
- Is a popular Container technology
- It is lightweight, portable and self-sufficient container



Docker - Container to Package Software



Virtual Machines vs Containers



Kubernetes / K8s

- Kubernetes is a *container orchestration tool* that manages applications/ services available on a container platform like Docker
- Developed by Google labs and later donated to CNCF (Cloud Native Computing Foundation)
- Open source
- Written in Golang
- Kubernetes manages the deployment, scaling, and operation of application containers.



kubernetes



Thank You!